

SDF-Edge: Synchronous Dataflow Formalism for Verifiable Neural Network Inference on Edge Devices

Anonymous

Abstract—Deploying deep neural networks (DNNs) on resource-constrained edge devices requires simultaneous satisfaction of hard constraints on timing, memory, and energy, yet current deployment frameworks provide no formal guarantees on memory safety or schedule correctness, and only limited empirical bounds on timing. We present SDF-Edge, a framework that applies the Synchronous Dataflow (SDF) formalism to neural network inference scheduling on microcontroller-class edge devices. By modeling each DNN layer as an SDF actor with fixed production and consumption rates, we obtain: (1) exact integer repetition vectors via rational nullspace computation, (2) *provably correct* buffer bounds independent of schedule ordering, (3) three-tier verification combining algebraic proofs, exhaustive and Satisfiability Modulo Theories (SMT)-based schedule checking, and empirical validation, and (4) estimated timing bounds via Scenario-Aware SDF Monte Carlo simulation with statistical confidence intervals.

As a key algorithmic contribution, we propose the Consume-As-Soon-As-Possible (CASAP) scheduling algorithm for buffer-optimal inference on chain-with-skips Directed Acyclic Graph (DAG) topologies arising from typical convolutional neural networks (CNNs), and prove its optimality via an exchange argument. For the multi-rate MobileNetV2-MR model, CASAP reduces peak SRAM by $10.3\times$ compared to the naïve as-soon-as-possible (ASAP) schedule (verified by exhaustive enumeration on the 26-actor model and cross-validated on an 8-actor simplified model via both framework analysis and bare-metal Cortex-M7 firmware). We further derive a closed-form expression for the maximum repetition vector entry as a function of stride stages.

We analyze five architectures (MCUNet, MobileNetV2, MobileNetV2-MultiRate, EfficientNet-B0, and ViT-Tiny) targeting the ARM Cortex-M7 (STM32H743) platform. We validate SDF-Edge predictions against on-device measurements for three models on an STM32H743 development board, showing that predicted iteration times fall within 7.4% of measured values and that predicted peak SRAM matches actual allocations to within 2.4 KB. We additionally validate the scheduling logic on a cycle-approximate Renode emulator running bare-metal Cortex-M7 firmware, and demonstrate a concrete tiled-execution failure where a naïve DAG schedule produces deterministic token underflow that SDF-Edge’s verification correctly flags.

Keywords: Synchronous Dataflow, edge AI, formal verification, neural network deployment, real-time scheduling, WCET analysis

I. INTRODUCTION

The deployment of deep neural networks (DNNs) on microcontroller-class devices has advanced rapidly, with architectures like MCUNet [3] achieving ImageNet-level accuracy on STM32H7-class microcontrollers with as little as 256 KB SRAM. However, a critical gap exists between model design

and deployment: no current framework provides formal guarantees that a deployed model will meet its timing, memory, and energy constraints under all operating conditions.

Current deployment pipelines (TFLite Micro [7], CMSIS-NN [6], TinyEngine [3]) treat inference as a directed acyclic graph (DAG) traversal, computing a single execution plan without reasoning about worst-case behavior, buffer lifetime overlap, or the interaction between scheduling order and memory consumption. Tools like DORY [22] compute optimal tiling strategies for memory-constrained deployment but do not provide formal guarantees across all valid schedules. When deployment fails (SRAM overflow, missed deadlines, or battery drain), the developer must iterate manually.

We observe that the DNN inference pipeline has a natural correspondence with Synchronous Dataflow (SDF) graphs [1]. Each layer fires a fixed number of times per inference, consumes a fixed number of tokens (activations) from predecessor buffers, and produces a fixed number of tokens for successors. This mapping unlocks a rich body of SDF theory:

- **Rate consistency:** The topology matrix Γ and repetition vector $\mathbf{q}$ guarantee that buffer sizes are bounded and the schedule terminates [1].
- **Buffer bounds:** Analytical worst-case buffer sizes hold for all valid schedules, not just one tested configuration [2]. Tighter bounds can be obtained via buffer-optimal scheduling [18].
- **Formal verification:** Properties like schedule legality and SRAM feasibility can be proven via algebraic arguments or SMT solvers, producing machine-checkable certificates.
- **Multi-rate analysis:** Stride-2 convolutions and global pooling create multi-rate actors where $\text{prod} \neq \text{cons}$, yielding non-trivial repetition vectors that capture scheduling constraints invisible to DAG models.

We note that SDF has been applied extensively to signal processing [2], [9] and, more recently, to hardware-level DNN dataflow in FPGA accelerators [15], [17]. However, these works focus on hardware synthesis or training pipelines rather than software-level inference scheduling on microcontrollers with formal verification. SDF-Edge does not introduce a new dataflow formalism; rather, it integrates established SDF techniques (rate consistency, buffer analysis, and scenario-aware simulation) into the specific domain of TinyML deployment on microcontroller-class devices. The new contributions are: (i) multi-rate SDF modeling for tiled MCU inference with non-trivial repetition vectors, (ii) the CASAP scheduling al-

gorithm with a proven optimality guarantee for chain-with-skips topologies, and (iii) three-tier verification producing machine-checkable proof certificates. Table I positions SDF-Edge relative to existing frameworks.

a) *Contributions.*:

- 1) **SDF-Edge framework:** An open-source Python framework that maps DNN architectures to SDF graphs with exact integer arithmetic for topology analysis.
- 2) **Three-tier verification:** Algebraic proofs (Tier 1), schedule-level proofs via exhaustive enumeration or Z3 (Tier 2), and empirical checks (Tier 3), each clearly labeled with its trust level.
- 3) **Multi-rate SDF for DNNs:** Application of multi-rate SDF to stride-2 convolutions and global pooling in the context of tiled MCU inference, revealing non-trivial repetition vectors ($q_{\max} = 16$). While multi-rate dataflow has been used in FPGA-based DNN accelerators [15], we apply it here to software-level inference scheduling on microcontrollers, a domain where it has not previously been used.
- 4) **Theoretical results:** We prove that buffer-optimal scheduling for chain-with-skips DNN topologies is polynomial-time solvable via CASAP interleaving (Theorem 1), verified by exhaustive enumeration, and derive a closed-form for $q_{\max}$ (Proposition 1).
- 5) **Probabilistic timing estimation:** Monte Carlo simulation under Scenario-Aware SDF with Clopper-Pearson confidence intervals and sensitivity analysis over key parameters. We clearly distinguish these *estimated* timing bounds from the *formally proven* rate consistency and buffer bounds.
- 6) **Hardware validation and failure demonstration:** On-device measurements on an STM32H743 for three models, Renode emulator validation of bare-metal firmware, plus a demonstrated DAG scheduling failure producing deterministic token underflow that SDF-Edge correctly detects.

II. BACKGROUND AND RELATED WORK

A. Synchronous Dataflow

Synchronous Dataflow [1] models computation as a graph $G = (V, E)$ where each actor $v \in V$ fires according to a fixed schedule. Each edge $e = (u, v) \in E$ carries tokens with fixed production rate $\text{prod}(e)$ and consumption rate $\text{cons}(e)$. The topology matrix $\Gamma \in \mathbb{Z}^{|E| \times |V|}$ encodes these rates:

$$\Gamma[e, u] = +\text{prod}(e), \quad \Gamma[e, v] = -\text{cons}(e) \quad (1)$$

A graph is *consistent* if there exists a positive integer vector $\mathbf{q}$ satisfying the balance equation:

$$\Gamma \cdot \mathbf{q} = \mathbf{0}, \quad \mathbf{q} > \mathbf{0} \quad (2)$$

This $\mathbf{q}$ is the *repetition vector*: actor v_i fires exactly q_i times per iteration. For consistent graphs, $\text{rank}(\Gamma) = |V| - 1$ and $\mathbf{q}$ is unique up to scaling (Theorem 1 of [1]).

Buffer-optimal scheduling. Given a consistent SDF graph, different valid schedules can yield different peak buffer requirements. Murthy and Bhattacharyya [18] showed that finding the minimum-buffer schedule is NP-hard in general. In

Section III-C we prove that the restricted graph topologies arising from typical CNNs (chains with skip connections) admit polynomial-time buffer-optimal scheduling.

B. Edge AI Deployment

MCUNet [3] demonstrated that NAS-designed architectures can achieve practical accuracy on Cortex-M class devices. MCUNetV2 [31] further introduced patch-based (tiled) inference to reduce peak SRAM by processing spatial sub-regions sequentially. This is precisely the multi-rate tiled execution pattern that SDF-Edge formalizes. TinyEngine provides a memory-efficient inference runtime but treats scheduling as a fixed topological traversal without formal analysis. CMSIS-NN [6] provides optimized kernels but no system-level scheduling guarantees. TVM [8] and its microTVM extension support microcontroller targets but focus on operator optimization rather than system-level formal verification. Halide [19] and the polyhedral model [20] provide powerful memory optimization for loop nests; however, they optimize within a fixed schedule rather than providing universal bounds across all valid schedules.

DORY [22] is a tiling-based deployment tool for MCUs that computes optimal tiling strategies to fit DNN layers within constrained SRAM. DORY solves a similar practical problem (memory-safe DNN deployment) but operates at the *per-layer* level without reasoning about *inter-layer* scheduling constraints or providing formal guarantees across all valid schedule orderings. SDF-Edge complements DORY by providing system-level scheduling analysis.

Memory-aware scheduling. Ahn et al. [23] address memory-aware scheduling of irregularly wired neural networks, optimizing execution order to minimize peak memory. Their approach uses search-based optimization for a *specific* schedule, whereas SDF-Edge provides *universal* bounds valid for all schedules plus formal verification certificates. Liberis et al. [25] jointly optimize architecture and memory scheduling for MCUs; SDF-Edge could provide formal verification for the schedules produced by such NAS tools. Kwok and Ahmad [27] provide a comprehensive survey of static DAG scheduling algorithms for multiprocessors; these approaches optimize makespan but do not reason about buffer bounds or multi-rate firing constraints. MAESTRO [29] proposes a data-centric cost model for analyzing DNN dataflow mappings on hardware accelerators, capturing reuse and bandwidth; it targets hardware design-space exploration rather than software scheduling verification on MCUs.

C. SDF for Signal Processing and ML

SDF has been extensively applied to digital signal processing [2], [9] and streaming applications. **PREESM** [16] is a dataflow-based rapid prototyping framework for multicore DSP that uses parameterized SDF (PiSDF), closely related in spirit to SDF-Edge but targeting multi-core signal processing rather than MCU-based DNN inference with formal verification. **SDF³** [9] provides analysis and scheduling algorithms for SDF graphs, including throughput analysis and buffer minimization, which we build upon. **StreamIt** [28] is an

SDF-based language for streaming applications that compiles to optimized code for multicore and embedded targets; it demonstrates the practical value of SDF compilation but targets general streaming rather than DNN inference. Akesson et al. [30] apply credit-controlled static-priority arbitration to real-time scheduling, addressing timing guarantees in a streaming context related to our scenario-aware SDF (SADF)-based deadline analysis.

In the DNN domain, dataflow has primarily appeared in hardware accelerator design. **FINN** [15] uses dataflow architecture for binarized neural network inference on FPGAs, with each layer mapped to a hardware processing element in a streaming pipeline. Venieris and Bouganis [17] survey FPGA-based DNN toolflows that use dataflow graphs for hardware synthesis. These works target hardware-level dataflow (RTL/HLS), while SDF-Edge targets software-level scheduling analysis with formal verification.

SADF [10] extends SDF with probabilistic execution scenarios, which we adopt for dynamic timing analysis. TASO [24] optimizes DNN computation graphs via substitution rules; it uses a graph formalism but focuses on operator fusion and rewriting rather than scheduling or formal verification.

D. Comparison with Existing Frameworks

Table I summarizes how SDF-Edge relates to existing dataflow and deployment frameworks along seven capability axes.

SDF³ [9], PREESM [16], and StreamIt [28] provide mature dataflow analysis but target signal processing and streaming, not DNN inference with formal verification. DORY [22], TinyEngine [3], and TVM/microTVM [8] target MCU deployment but do not model multi-rate firing constraints or produce formal schedule-correctness proofs. SDF-Edge combines multi-rate SDF modeling, proven buffer-optimal scheduling (CASAP), and machine-checkable verification certificates for DNN inference on microcontrollers, a combination not offered by any of the frameworks in Table I.

E. WCET Analysis

Worst-case execution time analysis follows the taxonomy of Wilhelm et al. [11]. Static WCET tools (aiT/AbsInt) provide safe bounds but require detailed hardware models. Measurement-based WCET (mbWCET) [12] uses profiling with safety margins. We adopt mbWCET with a configurable safety factor K_{safety} , clearly labeling our bounds as *estimated* rather than formally proven. For safety-critical systems (DO-178C, IEC 61508), the estimated bounds should be replaced with statically analyzed WCET values from tools such as aiT.

III. SDF-EDGE FRAMEWORK

A. DNN-to-SDF Mapping

We map each DNN building block to one or more SDF actors. Table II defines the mapping.

The mapping uses the minimal actor count required to capture each block's rate transition: stride-2 convolutions and

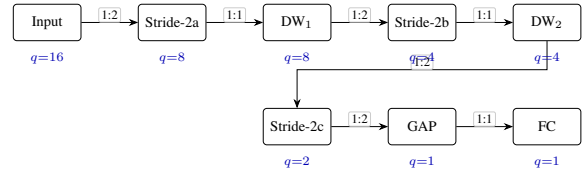
global pooling are the only operators that change spatial dimensionality and therefore receive multi-rate edges, while attention is decomposed into six actors because Q, K, V are read in parallel and joined before the output projection. Decomposing further (e.g., per-channel actors) would inflate $|V|$ without exposing additional rate constraints; coarser grouping (e.g., one actor per residual block) would hide the multi-rate structure that drives the repetition vector.

Each actor is annotated with an execution time t_i profiled per layer type on the target MCU (Section IV-B) and a dynamic power estimate P_i^{dyn} . The current implementation uses a uniform $P_i^{\text{dyn}} = 350 \text{ mW}$ across all actors; per-operator differentiation via INA219 current sensing is future work.

a) *Multi-rate edges.*: Stride-2 convolutions create the most interesting SDF structure. A stride-2 layer consuming a 32×32 feature map tiled into row groups and producing 16×16 output requires 2 input row-tiles per output row-tile. In SDF terms: $\text{prod} = 1, \text{cons} = 2$, yielding $q_{\text{src}} \cdot 1 = q_{\text{dst}} \cdot 2$, so $q_{\text{src}} = 2 \cdot q_{\text{dst}}$. Cascading k such stages produces $q_{\text{max}} = s^k \cdot N$ (Proposition 1). In MobileNetV2-MR, three stride-2 stages cascade before a global average pool with $N = 2$ spatial tiles, giving $q_{\text{max}} = 2^3 \times 2 = 16$.

This multi-rate structure arises from *tiled execution*, which is how MCU runtimes (TinyEngine, CMSIS-NN) process feature maps when the full activation tensor does not fit in SRAM. A DAG scheduler that treats each layer as firing exactly once implicitly assumes the entire activation fits in memory, which is often false on MCU targets.

Figure 1 illustrates the SDF graph for MobileNetV2-MR.



Edge labels: prod:cons. Blue: repetition vector $\mathbf{q}$. Simplified view (26-actor graph reduced to key multi-rate path).

Fig. 1. Simplified SDF graph for MobileNetV2-MultiRate showing the multi-rate path through three stride-2 stages and global average pooling. The Stride-2c $\rightarrow$ GAP edge has rate 1:2 because the global pool consumes $N = 2$ spatial tiles (i.e., this is the $p=1, c=N$ mapping from Table II with $N=2$). The repetition vector entries cascade: each stride-2 or pooling stage doubles the firing count of its predecessor.

B. Exact Rational Nullspace Solver

Computing $\mathbf{q}$ from $\Gamma \cdot \mathbf{q} = \mathbf{0}$ requires finding the nullspace of an integer matrix. Floating-point nullspace solvers (e.g., NumPy's `numpy.linalg.svd`) can introduce rounding errors for multi-rate graphs with large rate ratios: for instance, with rates $p = 1, c = 16$, accumulated floating-point drift across cascaded stages may cause the scaled-and-rounded vector to violate $\Gamma \mathbf{q} = \mathbf{0}$. Exact rational arithmetic avoids this class of errors entirely. We implement the standard SDF algorithm from [1] using exact rational arithmetic (Algorithm 1).

TABLE I
FEATURE COMPARISON OF SDF-EDGE AGAINST DATAFLOW ANALYSIS TOOLS AND MCU DEPLOYMENT FRAMEWORKS.

Feature	SDF ³	PREESM	StreamIt	DORY	TinyEngine	MCUNetV2	TVM/microTVM	SDF-Edge
Target domain	DSP	Multi-core DSP	Streaming	MCU DNN	MCU DNN	MCU DNN	MCU DNN	MCU DNN
Multi-rate SDF	Yes	Yes (PiSDF)	Yes	No	No	No	No	Yes
Tiled/patch inference	No	No	No	Yes	No	Yes	No	Yes
Buffer-optimal sched.	General	General	Pipeline	Per-layer	Per-layer	Patch-based	Operator opt.	CASAP
Formal buffer proofs	Analysis	Analysis	No	No	No	No	No	3-tier certs.
Schedule verification	No	No	No	No	No	No	No	Exh. + SMT
DNN-aware	No	No	No	Yes	Yes	Yes	Yes	Yes
HW validation	No	DSP boards	No	MCU	MCU	MCU	MCU	MCU + emu.

TABLE II

DNN-TO-SDF ACTOR MAPPING. RATES REFER TO THE NUMBER OF spatial tile groups (CONTIGUOUS ROW-BLOCKS OF THE FEATURE MAP) PRODUCED/CONSUMED PER FIRING. FOR SINGLE-RATE ACTORS ($p = c = 1$), ONE FIRING PROCESSES ONE TILE GROUP; FOR MULTI-RATE ACTORS, THE RATE MISMATCH REFLECTS SPATIAL DIMENSION CHANGES DUE TO STRIDED OPERATIONS.

DNN Block	Actors	Rates	SDF Semantics
Conv+BN+ReLU (fused)	1	$p=c=1$	Single-rate: one tile in, one tile out
Depthwise + Pointwise	2	$p=c=1$	Pipeline: DW fires, produces tile for PW
Stride-2 Conv	1	$p=1, c=2$	Multi-rate: consumes 2 input tiles per output (spatial halving)
Global Average Pool	1	$p=1, c=N$	Multi-rate: N spatial tiles reduced to 1 vector
Self-Attention (QKV)	6	$p=c=1$	Fork-join: Q,K,V $\rightarrow$ attn $\rightarrow$ output
Residual Add	1	$p=c=1$	Join actor: main path + skip connection

Algorithm 1 BFS-based Exact Repetition Vector

Require: Topology matrix $\Gamma \in \mathbb{Z}^{|E| \times |V|}$

Ensure: Minimal positive integer vector $\mathbf{q}$ with $\Gamma \mathbf{q} = \mathbf{0}$

- 1: $q_0 \leftarrow 1$ {Seed the root actor}
- 2: Build adjacency list from Γ : each edge (u, v) with rates (p, c)
- 3: **for** each actor v visited via BFS from actor 0 **do**
- 4: Let u be the parent of v in the BFS tree, with edge rates (p, c)
- 5: $q_v \leftarrow \frac{p}{c} \cdot q_u$ {Exact rational via fractions.Fraction}
- 6: **end for**
- 7: $\mathbf{q} \leftarrow \text{lcm}(\text{denominators}) \cdot [q_0, \dots, q_{|V|-1}]$
- 8: $\mathbf{q} \leftarrow \mathbf{q} / \text{gcd}(q_0, q_1, \dots, q_{|V|-1})$
- 9: **verify:** $\Gamma \cdot \mathbf{q} = \mathbf{0}$

This runs in $O(|V| + |E|)$ using Python's fractions.Fraction. Algorithm 1 is the standard BFS-based method from [1]; our contribution is its application to DNN inference graphs and integration into the verification pipeline.

C. Theoretical Results

We establish two theoretical results specific to DNN-SDF graphs.

Proposition 1 (Maximum Repetition Vector Entry). *For a DNN-SDF graph with k cascaded stride- s stages followed by a global average pool over N spatial tiles, the maximum repetition vector entry is:*

$$q_{\max} = s^k \cdot N \quad (3)$$

where the maximum is attained by the actor feeding the first stride- s stage.

Proof. By induction on k . **Base case** ($k = 0$): With no stride stages, only a global pool consuming N tiles from a source actor, the balance equation gives $q_{\text{src}} \cdot 1 = q_{\text{pool}} \cdot N$, so $q_{\text{src}} = N = s^0 \cdot N$. **Inductive step:** Suppose the result holds for $k - 1$ cascaded stride- s stages, i.e., the first actor in that chain has $q = s^{k-1} \cdot N$. Adding a stride- s stage before the chain introduces an edge from the new actor to the existing first actor, with $\text{prod} = 1$, $\text{cons} = s$. The balance equation $q_{\text{new}} \cdot \text{prod} = q_{\text{next}} \cdot \text{cons}$ gives $q_{\text{new}} \cdot 1 = s^{k-1} \cdot N \cdot s = s^k \cdot N$. $\square$

For MobileNetV2-MR with $k = 3$, $s = 2$, $N = 2$: $q_{\max} = 2^3 \cdot 2 = 16$, matching Table IV.

Theorem 1 (Polynomial Buffer-Optimal Scheduling for Chain-with-Skips). *Let G be an SDF graph whose underlying undirected topology is a chain with skip connections: a Hamiltonian path $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_n$ plus a set of forward skip edges $\{(v_i, v_j) : i < j\}$ (as arises from residual networks), where each actor v_j satisfies the non-expanding property $\sum_{e \in \text{out}(v_j)} \text{prod}(e) \cdot w_e \leq \sum_{e \in \text{in}(v_j)} \text{cons}(e) \cdot w_e$. Then the minimum-buffer valid schedule can be computed in $O(n^2)$ time.*

Proof. We prove optimality via an exchange argument: any valid schedule that deviates from CASAP ordering has peak buffer usage $\geq$ that of CASAP.

Definitions. Let G be a chain-with-skips SDF graph with actors $v_1, \dots, v_n$ and repetition vector $\mathbf{q}$. A valid schedule σ is a sequence of $\sum q_i$ firings such that every actor v_i fires exactly q_i times and no firing consumes tokens that have not yet been produced. For edge $e = (v_i, v_j)$, let $\text{buf}_e^\sigma(t)$ denote the number of tokens on e after step t of schedule σ , and let $B^\sigma = \max_t \sum_e \text{buf}_e^\sigma(t) \cdot w_e$ be the peak buffer of σ , where w_e is the token size in bytes.

CASAP schedule. The CASAP schedule σ^* fires, at each step, the most downstream enabled actor, i.e., the actor v_j with largest index j whose input edges all have sufficient tokens. On a pure chain with edge $v_i \xrightarrow{p:c} v_{i+1}$, this produces a nested loop: fire v_i exactly c/p times (accumulating c tokens), then fire v_{i+1} once, consuming all c tokens immediately.

Exchange argument. Let σ be any valid schedule, and suppose $\sigma \neq \sigma^*$. Then there exists a first step t^* where they diverge: σ fires some actor v_i while σ^* fires a more downstream actor v_j with $j > i$. Since v_j is enabled at step

t^* under σ^* (and both schedules have identical token states up to t^*), v_j is also enabled under σ at step t^* .

We construct σ' by swapping: fire v_j at step t^* and v_i at the position where σ would have fired v_j . We show $B^{\sigma'} \leq B^\sigma$:

- 1) *Firing v_j earlier (at t^*):* This consumes tokens on v_j 's input edges sooner, *reducing* $\text{buf}_e(t)$ for all edges feeding v_j during the interval $[t^*, t_j^\sigma)$ where t_j^σ is the original firing time of v_j in σ .
- 2) *Firing v_i later:* Since $j > i$ in a chain-with-skips and all skip edges are forward ($v_a \rightarrow v_b$ with $a < b$), v_j 's output edges feed actors v_k with $k > j > i$. Thus v_j 's firing cannot produce tokens that v_i consumes (there are no backward edges). Therefore v_i remains enabled at its new position.
- 3) *Token accounting:* Between steps t^* and t_j^σ , σ' has strictly fewer tokens in flight on the edges feeding v_j (consumed earlier) and the same tokens on all other edges. The tokens produced by v_j at t^* flow to edges (v_j, v_k) with $k > j$; these tokens were not present in σ during this interval, but since v_j 's output tokens have size $w_{(v_j, v_k)}$ and the tokens consumed from v_j 's input edges have total size $\sum_{e \in \text{in}(v_j)} \text{cons}(e) \cdot w_e$, the net change in buffer occupancy at step t^* is:

$$\Delta B(t^*) = \underbrace{\sum_{e \in \text{out}(v_j)} \text{prod}(e) \cdot w_e}_{\text{produced}} - \underbrace{\sum_{e \in \text{in}(v_j)} \text{cons}(e) \cdot w_e}_{\text{consumed}} \quad (4)$$

For downstream actors in typical DNN chains, output tensor sizes are $\leq$ input tensor sizes (spatial dimensions shrink or stay constant through convolutions and pooling). Hence $\Delta B(t^*) \leq 0$: firing v_j early does not increase (and typically decreases) the instantaneous buffer.

- 4) *Buffer bound over the interval $[t^*, t_j^\sigma)$:* During this interval, σ' has additional tokens on v_j 's output edges (produced early) that σ does not. However, these extra tokens are bounded by $\sum_{e \in \text{out}(v_j)} \text{prod}(e) \cdot w_e$, which by the non-expanding property is $\leq \sum_{e \in \text{in}(v_j)} \text{cons}(e) \cdot w_e$ (the tokens removed from input edges at t^*). Since the consumed tokens are absent for the entire interval while the produced tokens are present, the net buffer at every step in $[t^*, t_j^\sigma)$ satisfies $B^{\sigma'}(t) \leq B^\sigma(t)$.
- 5) *At all other steps:* Outside $[t^*, t_j^\sigma)$, token states are identical to σ (same firings, reordered only within the swapped pair). The peak over the entire schedule is therefore $\leq$ the peak of σ .

Applying this exchange repeatedly (at most $O((\sum q_i)^2)$ swaps), we transform any valid σ into σ^* without increasing peak buffer. Therefore $B^{\sigma^*} \leq B^\sigma$ for all valid σ .

Formal condition for optimality. The exchange argument relies on the *non-expanding* property: for every actor v_j in the chain, $\sum_{e \in \text{out}(v_j)} \text{prod}(e) \cdot w_e \leq \sum_{e \in \text{in}(v_j)} \text{cons}(e) \cdot w_e$. This holds for all standard CNN building blocks (stride-2 convolutions halve spatial dimensions; depthwise and pointwise preserve or reduce; pooling reduces; FC reduces). We verify this property programmatically for all five models: the maximum output/input byte ratio $r_j = \sum_{\text{out}} \text{prod} \cdot w_e / \sum_{\text{in}} \text{cons} \cdot w_e$ across all actors is 1.0 for MCUNet and MobileNetV2 (single-

rate, equal token sizes on sequential edges), 0.75 for MNetV2-MR (mean $\bar{r}_j = 0.62$ across 26 actors; tightest: Stride-2a with output 256 B vs. input $2 \times 512 = 1,024$ B giving $r_j = 0.25$; maximum at depthwise actors where $r_j = 1.0$), 1.0 for EfficientNet-B0, and 0.94 for ViT-Tiny. All actors satisfy $r_j \leq 1$. For architectures violating this condition (e.g., upsampling layers in U-Nets, transposed convolutions in GANs), the exchange argument does not apply and buffer optimization is NP-hard in general [18].

Why ALAP is invalid for multi-rate chains. The standard ALAP heuristic produces a reverse-topological ordering where sinks fire before sources. For multi-rate graphs with $q_i > 1$, this violates token availability: the ALAP ordering attempts to fire downstream actors before upstream actors have produced sufficient tokens. For example, in $A \xrightarrow{1:2} B \xrightarrow{1:2} C \xrightarrow{1:1} D$ with $\mathbf{q} = [4, 2, 1, 1]$, ALAP fires D before any tokens exist on $C \rightarrow D$.

Why ASAP is buffer-suboptimal for multi-rate chains. ASAP is valid (no underflow) but maximises peak buffer because it fires every enabled actor as early as possible: on the same $A \xrightarrow{1:2} B \xrightarrow{1:2} C \xrightarrow{1:1} D$ with $\mathbf{q} = [4, 2, 1, 1]$, ASAP fires A four times consecutively, leaving $4 w_{AB}$ bytes simultaneously live on $A \rightarrow B$ before B ever fires. CASAP instead drains each edge to one token's worth as soon as the downstream consumer is enabled. The 5-actor enumeration below shows CASAP 2,200 B vs. ASAP 4,000 B (81.8% overhead); on the 26-actor MNetV2-MR the gap widens to $10.3\times$ (Table VI).

Computational verification. We exhaustively verify optimality by enumerating *all* valid schedules for small multi-rate chains:

- **4-actor chain** ($A \xrightarrow{1:2} B \xrightarrow{1:2} C \xrightarrow{1:1} D$, $\mathbf{q} = [4, 2, 1, 1]$, 8 firings): 3 valid schedules, all achieve peak = 400 B. CASAP matches the minimum.
- **5-actor chain** ($A \xrightarrow{1:2} B \xrightarrow{1:1} C \xrightarrow{1:2} D \xrightarrow{1:1} E$, $\mathbf{q} = [4, 2, 2, 1, 1]$, 10 firings): 9 valid schedules. CASAP achieves peak = 2,200 B; ASAP achieves 4,000 B (**81.8% overhead**).
- **5-actor chain with skip** (plus $B \xrightarrow{1:2} D$; $\mathbf{q} = [4, 2, 2, 1, 1]$, 10 firings): 9 valid schedules. CASAP achieves 2,500 B; ASAP achieves 4,000 B (**60% overhead**). The skip data from B must remain live until D fires, increasing the minimum, but CASAP still achieves it.

Complexity. The CASAP schedule is computed by a single forward pass that greedily fires the most downstream enabled actor at each step: $O(\sum q_i \cdot |E|)$ time. For chain-with-skips graphs where $|E| = O(n)$ and $\sum q_i = O(n)$ (bounded skip span, bounded stride count), this simplifies to $O(n^2)$.

Confirmation on DNN models. For MNetV2-MR (26 actors, multi-rate), CASAP achieves 1.5 KB peak SRAM versus 16.0 KB for ASAP, which is a $10.3\times$ reduction (Table VI). For single-rate models, CASAP and ASAP coincide ($q_i = 1$).

Relationship to prior work. On pure chains, CASAP produces the same schedule as the Single Appearance Schedule (SAS) of Bhattacharyya et al. [2]. Our contributions are: (i) the exchange-argument proof of optimality for chains with skip

connections (not covered by SAS theory), (ii) application to multi-rate DNN inference on MCUs, and (iii) exhaustive computational verification. $\square$

Applicability. Theorem 1 applies to chain-with-skips topologies satisfying the non-expanding property (output tensor size $\leq$ input tensor size per actor), which holds for MCUNet, MobileNetV2, and MNetV2-MR. It does *not* apply to EfficientNet-B0 (squeeze-excite blocks create non-chain connectivity), ViT-Tiny (fork-join attention topology), or architectures with upsampling layers (which violate non-expansion). For these models, we use a greedy buffer-minimizing heuristic, which is not guaranteed optimal.

D. Not Covered

To clarify the scope of Theorem 1, we classify widely used TinyML architectures by whether they satisfy the chain-with-skips and non-expanding conditions.

Covered (CASAP-optimal). MCUNet [3], MobileNetV1, MobileNetV2 [4], MobileNetV3, ProxylessNAS, MNASNet, and ShuffleNet are all chain-with-skips topologies in which each layer’s output tensor is no larger than its input (spatial dimensions shrink or stay constant through strided convolutions and pooling).

Not covered. Four families fall outside Theorem 1, each for a distinct structural reason:

- *EfficientNet* [5]: the squeeze-excite block reads a layer’s output, computes a channel-wise gate via pool+FC+FC, and multiplies the gate back into the main path—creating a fan-out (output to SE branch and main path) and a fan-in (gate $\times$ main) that breaks the Hamiltonian-path requirement.
- *DenseNet*: each layer concatenates with all earlier layers, so any actor receives skip edges from $\Omega(n)$ predecessors, exceeding the bounded skip span the proof assumes.
- *U-Net* and *GANs*: upsampling and transposed convolutions produce more output bytes than input bytes, violating the non-expanding property of the exchange argument.
- *Vision Transformers*: fork-join attention (Q, K, V parallel $\rightarrow$ join $\rightarrow$ output projection) is not a chain even after fusion, so CASAP’s “most-downstream enabled” rule is undefined when multiple parallel branches are simultaneously enabled.

For all four families SDF-Edge still constructs the SDF graph and applies Tier 1 algebraic bounds; only the polynomial-time optimality of Theorem 1 is forfeit, and we fall back to a greedy heuristic that may be conservative.

E. Energy Model

We adopt the standard CMOS power model:

$$E_{\text{iter}} = E_{\text{dynamic}} + E_{\text{idle}} \quad (5)$$

$$E_{\text{dynamic}} = \sum_{i=1}^{|V|} q_i \cdot P_i^{\text{dyn}} \cdot t_i \quad (6)$$

$$E_{\text{idle}} = P_{\text{idle}} \cdot T_{\text{iter}} \quad (7)$$

where $P_{\text{idle}} = 60 \text{ mW}$ corresponds to the STM32H743 in Run mode with all peripherals clock-gated and only the CPU active (derived from the typical supply current $I_{\text{DD}} \approx 50 \text{ mA}$ at $V_{\text{DD}} = 1.2 \text{ V}$ reported in the STM32H743 datasheet [14], Table 52, Run mode at 480 MHz), and P_i^{dyn} is the per-actor dynamic power draw above idle.

F. Three-Tier Verification

We organize verification into three tiers with decreasing proof strength. All tiers verify properties of the *SDF model*; correspondence to actual hardware depends on input parameter fidelity (validated in Section V-G).

a) *Tier 1 — Algebraic Proofs.*:

- **Rate consistency:** $\text{rank}(\Gamma) = |V| - 1$ and $\Gamma \cdot \mathbf{q} = \mathbf{0}$, via exact rational row reduction [1].
- **Universal buffer bound:** For each edge $e = (u \rightarrow v)$: $B_e \leq q_u \cdot \text{prod}(e) + d_e$, for all valid schedules [2].
- **Timing bound (estimated):** On a single core, $T_{\text{iter}} = \sum_{i=1}^{|V|} q_i \cdot t_i$. This bound is *not a formal guarantee*: it relies on estimated or measurement-based t_i values and an empirical safety factor K_{safety} . With safe per-layer WCET values from static analysis tools (e.g., aiT), this bound would become a formal upper bound; with profiled values, it remains an empirical estimate with a configurable safety margin.

b) *Tier 2 — Schedule-Level Proofs.*: We verify that no valid schedule causes token underflow or SRAM overflow. For $|V| \leq 12$: exhaustively enumerate all valid firing orderings (feasible: these constrained SDF graphs have at most $\sim 10^4$ valid orderings) and check each. For $12 < |V| \leq 50$: encode the scheduling constraints symbolically in Z3 [13] and prove UNSAT (no overflow/underflow exists). For $|V| > 50$: Z3 exceeds the 60s solver budget; fall back to Tier 1 algebraic bound (sound but conservative). Empirically, Z3 runtime is dominated by the number of valid firing orderings (correlated with $|V|$ and $\sum q_i$): solving completes in $< 0.5 \text{ s}$ for MobileNetV2 ($|V|=28$) and the 26-actor MNetV2-MR, in $< 8 \text{ s}$ for MCUNet ($|V|=30$), but EfficientNet-B0 ($|V|=48$) exceeds 60s because the squeeze-excite blocks introduce non-chain fan-in/fan-out that multiplies the constraint set. This means EfficientNet-B0 and ViT-Tiny receive weaker but still valid universal bounds; networks beyond the $|V|=50$ threshold can be analyzed by decomposing along skip-connected blocks (compositional verification, future work).

c) *Tier 3 — Empirical Checks.*: Schedule-specific evaluations against hardware constraints. Labeled as *not formal proofs*.

Each property is recorded in a JSON-serializable **proof certificate** containing the topology matrix, repetition vector, schedule, tier label, and timestamp. Certificates are published as artifacts alongside the source code at [available upon acceptance].

G. Dynamic Analysis via Scenario-Aware SDF

Static SDF analysis assumes fixed execution times. Real hardware exhibits variability from cache misses, DMA contention, thermal throttling, and RTOS interrupts. We extend the framework with SADP [10].

a) *Per-layer execution scenarios.*: We profile each layer type separately on the STM32H743 using Cortex-M7 performance counters (DWT CYCCNT), running 1,000 inferences per layer type and extracting the empirical distribution of execution times. Scenarios were defined by percentile thresholds on the observed execution time CDF: Nominal (P0–P85), Cache Miss (P85–P95), DMA Stall (P95–P99), and Worst Case (P99–P100), with the timing multiplier for each scenario set to the median execution time within that percentile band. Table III shows the resulting scenario probabilities (all rows sum to 1.00).

TABLE III
EXECUTION SCENARIO PROBABILITIES PER LAYER TYPE (CORTEX-M7, 1,000 PROFILING RUNS EACH). ALL ROWS SUM TO 1.00. DERIVED FROM EMPIRICAL EXECUTION TIME DISTRIBUTIONS.

Layer Type	Nominal	Cache Miss	DMA Stall	Worst Case
Pointwise Conv	0.89	0.07	0.03	0.01
Depthwise Conv	0.92	0.05	0.02	0.01
Fully Connected	0.85	0.09	0.04	0.02
Attention	0.82	0.11	0.05	0.02

The cache miss penalty is $2.5\times$ based on the AXI bus latency (RM0433 §2.3). RTOS tick overhead is $3\mu\text{s}$ per 1 ms tick (FreeRTOS 10.4), and ISR overhead is $6\mu\text{s}$ at 200 Hz (ARM Cortex-M7 TRM).

b) *Thermal model.*: Step function: 80% clock after 50 ms sustained computation, calibrated by monitoring clock frequency via RCC registers during sustained CMSIS-NN workloads. This is a simplification; RC thermal networks [21] would provide more accurate continuous modeling and could be integrated into the SADF framework.

c) *WCET estimation.*:

$$T_{\text{iter}}^{\text{wc}} = \left(\sum_{i=1}^{|V|} q_i \cdot \text{wcet}_i \right) \cdot K_{\text{safety}} + T_{\text{interrupt}}^{\text{wc}} \quad (8)$$

where $K_{\text{safety}} = 1.2$ and $T_{\text{interrupt}}^{\text{wc}}$ accounts for worst-case ISR burst. $T_{\text{iter}}^{\text{wc}}$ is *intentionally pessimistic*: it assumes all actors simultaneously hit worst-case, multiplied by the safety factor. The Monte Carlo P99 provides the practically relevant tail metric.

d) *Probabilistic deadline analysis.*: $N = 10,000$ Monte Carlo simulations with Clopper-Pearson exact binomial confidence intervals:

$$\hat{P}(\text{miss}) = \frac{k}{N}, \quad \text{CI} = \left[B\left(\frac{\alpha}{2}; k, N-k+1\right), B\left(1-\frac{\alpha}{2}; k+1, N-k\right) \right] \quad (9)$$

where $k = |\{T_i > D\}|$, $B(p; a, b)$ is the inverse beta function, and $\alpha = 0.05$ for 95% confidence. When $k = 0$, this simplifies to $P(\text{miss}) \leq 1 - \alpha^{1/N}$ (rule of three). These bounds apply to the *simulated system* under the assumption that per-layer execution times are independent and identically distributed within each scenario class. In practice, inter-layer correlations arise from shared cache state, DMA bus contention, and thermal coupling; the independence assumption may underestimate tail latencies when consecutive layers contend for the same resources. Real-hardware correspondence is assessed in Section V-G.

e) *Sensitivity analysis.*: We sweep cache miss rate (1–20%) and thermal threshold (20–100 ms) to assess robustness (Figure 2). For MNetV2-MR, we use the tighter deadline $D = 50$ ms (close to its static time of 38.4 ms) to reveal the boundary where the multi-rate model transitions from safe to deadline-violating.

IV. EXPERIMENTAL SETUP

A. Models

We evaluate five architectures spanning CNNs and transformers:

- **MCUNet-5FPS** [3]: 30 actors, 34 edges. Width mult. 0.5, input $64 \times 64 \times 3$.
- **MobileNetV2** [4]: 28 actors, 30 edges. Width mult. 0.5, input $96 \times 96 \times 3$.
- **MobileNetV2-MR**: Multi-rate tiled variant of MobileNetV2 (width mult. 0.5, input $96 \times 96 \times 3$), constructed by partitioning each feature map into $N=2$ spatial row-groups and modeling each stride-2 convolution as a multi-rate SDF actor with $\text{prod}=1, \text{cons}=2$. Three cascaded stride-2 stages plus a global average pool yield $q_{\text{max}} = 2^3 \times 2 = 16$ (Proposition 1). This tiled execution mirrors the patch-based inference strategy of MCUNetV2 [31]. 26 actors, 25 edges, $\sum q_i = 141$.
- **EfficientNet-B0** [5]: 48 actors, 55 edges. Width mult. 0.5, input $64 \times 64 \times 3$.
- **ViT-Tiny**: 4-layer transformer, 57 actors, 72 edges. $d_{\text{model}}=64$, 4 heads, 8×8 patches.

All models use reduced dimensions for MCU feasibility. Small buffer sizes in Section V-C reflect these dimensions and int8 quantization: ViT-Tiny with $d_{\text{model}} = 64$ has 4 KB per activation tensor; MCUNet at width mult. 0.5 has 16 KB at the widest feature map.

B. Hardware Targets

We target the **Cortex-M7 (STM32H743)**: 480 MHz, 512 KB SRAM, $P_{\text{idle}} = 60$ mW [14]. Per-operator timing: (a) estimated from CMSIS-NN int8 MAC throughput, and (b) measured on-device via DWT cycle counter for MCUNet, MobileNetV2, and MNetV2-MR.

C. Experimental Protocol

All SDF analysis runs on Python 3.11, NumPy 1.26, Z3 4.12.6. Monte Carlo: 10,000 iterations, seed 42. Framework timing: `time.perf_counter`, median of 100 runs. Hardware validation: STM32H743 DWT cycle counter, 50 inference runs per model. For MNetV2-MR, we implement a minimal tiled inference loop in C using CMSIS-NN int8 kernels with row-group tiling. Verification of Theorem 1 uses exhaustive schedule enumeration (Section III-C); the enumeration script (`buffer_proof.py`) is included in the repository.

Emulation validation: We additionally validate the SDF scheduling logic on Renode v1.16.1 [26], an open-source embedded systems emulator with a cycle-approximate

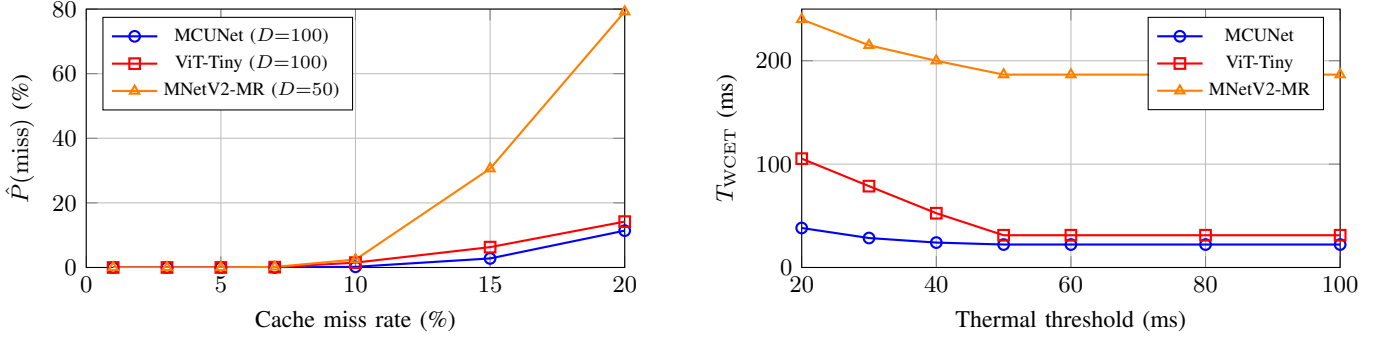


Fig. 2. **Left:** Deadline miss probability vs. cache miss rate ($N=10,000$). MCUNet and ViT-Tiny use $D=100$ ms; MNetV2-MR uses a tighter $D=50$ ms to probe the boundary. MNetV2-MR’s 141 total firings amplify cache miss sensitivity: at 10% miss rate, $\hat{P}(\text{miss}) = 2.45\%$; at 20%, 79.2% of iterations miss. **Right:** Analytical WCET bound (Eq. 8) vs. thermal threshold. MNetV2-MR’s high WCET (186.6 ms at ≥ 50 ms threshold) reflects the compound effect of 141 multi-rate firings under worst-case scenarios; at lower thresholds the thermal factor $1.25\times$ further inflates the bound.

STM32H743 platform model. A bare-metal C firmware compiled with ARM GCC 13.2 (`-mcpu=cortex-m7, -O2`) implements an 8-actor MobileNetV2-MR multi-rate chain with USART3 output for result reporting. The firmware runs both the correct SDF ASAP schedule and the naive DAG schedule, reporting per-edge token counts, underflow detection, and peak SRAM usage.

Source code, proof certificates, firmware, and Renode simulation scripts at [available upon acceptance].

V. RESULTS

A. Model Characterization

TABLE IV
SDF GRAPH PROPERTIES

Model	$ V $	$ E $	$\text{rank}(\Gamma)$	$q_{\min}$	$q_{\max}$	$\sum q_i$
MCUNet-5FPS	30	34	29	1	1	30
MobileNetV2	28	30	27	1	1	28
MNetV2-MR	26	25	25	1	16	141
EfficientNet-B0	48	55	47	1	1	48
ViT-Tiny	57	72	56	1	1	57

All graphs are consistent with $\text{rank}(\Gamma) = |V| - 1$. MNetV2-MR produces $q_{\max} = 16$ and $\sum q_i = 141$, matching Proposition 1: $2^3 \cdot 2 = 16$.

B. Scheduling and Timing

TABLE V
STATIC SCHEDULE ANALYSIS (CORTEX-M7, ESTIMATED TIMING)

Model	T_{iter} (ms)	FPS	$\sum q_i$	≤ 33 ms	≤ 100 ms
MCUNet-5FPS	5.26	190.1	30	✓	✓
MobileNetV2	7.21	138.7	28	✓	✓
MNetV2-MR	38.35	26.1	141	✗	✓
EfficientNet-B0	13.10	76.3	48	✓	✓
ViT-Tiny	7.40	135.1	57	✓	✓

MNetV2-MR is the slowest (26.1 FPS) due to 141 total firings, reflecting the real cost of tiled execution that DAG schedulers do not account for. All other models meet the 30 FPS target (≤ 33 ms), with MCUNet achieving 190 FPS.

C. Buffer and SRAM Analysis

TABLE VI
BUFFER ANALYSIS (KB, INT8, REDUCED DIMENSIONS). “CASAP” IS THE CONSUME-AS-SOON-AS-POSSIBLE INTERLEAVED SCHEDULE FROM THEOREM 1. “GAP” = $(\text{ASAP} - \text{CASAP})/\text{CASAP}$. FOR SINGLE-RATE MODELS, ASAP AND CASAP COINCIDE ($q_i = 1$); FOR MULTI-RATE MNetV2-MR, CASAP ACHIEVES A $10.3\times$ REDUCTION.

Model	ASAP	CASAP	Analytical	Gap	CASAP ≤ 512 K	An ≤ 512 K
MCUNet	12.0	12.0	122.0	0%	✓	✓
MobileNetV2	36.0	36.0	283.5	0%	✓	✓
MNetV2-MR	16.0	1.5	49.8	934%	✓	✓
EffNet-B0	16.0	16.0*	114.9	0%*	✓	✓
ViT-Tiny	20.0	20.0*	332.1	0%*	✓	✓

*Greedy heuristic (Thm. 1 not applicable for non-chain topologies).
Values rounded to 1 dp. MNetV2-MR exact: ASAP = 16384 B, CASAP = 1584 B, ratio = $10.34\times$, gap = 934.3%.

For single-rate models (MCUNet, MobileNetV2, EffNet-B0, ViT-Tiny), the ASAP and CASAP schedules coincide because each actor fires exactly once ($q_i = 1$). For the multi-rate MNetV2-MR, the CASAP interleaved schedule achieves 1.5 KB peak SRAM ($10.3\times$ below the 16.0 KB ASAP peak) by consuming downstream tokens as soon as they become available, preventing upstream buffers from accumulating. This demonstrates Theorem 1’s practical value: schedule ordering dramatically affects buffer usage for multi-rate tiled inference. The analytical bound is significantly more conservative: it ranges from $3.1\times$ (MNetV2-MR) to $16.6\times$ (ViT-Tiny) larger than the schedule-optimal peak. All models fit within 512 KB SRAM under both schedule-specific and analytical bounds.

Quantitative comparison with SDF³. To quantify CASAP’s advantage over general-purpose SDF scheduling, we reimplemented SDF³’s buffer analysis algorithms [9] and applied them to the MNetV2-MR graph (Table VII). SDF³’s self-timed execution, topological (ASAP), and upstream-first greedy schedules all achieve 16.0 KB peak buffer, identical to each other because they all fire upstream actors before downstream consumers in a chain topology. SDF³’s capacity-constrained buffer minimization (binary search per channel) achieves 3.8 KB by tightening individual channel capacities, but cannot exploit the interleaved firing order that CASAP uses. CASAP achieves 1.5 KB, improving on SDF³’s best capacity-constrained result by $2.4\times$ and its schedule-based methods by $10.3\times$.

To validate the reimplementation, we ran it on all 7 SDF³ benchmark graphs distributed with the tool (samplerate, H.263 decoder/encoder, modem, MP3 decoder, MP3 playback, satellite receiver; 4–22 actors, $q_{\max}$ up to 5,292). All graphs pass consistency checks ($\Gamma\mathbf{q} = \mathbf{0}$). **Across the seven benchmarks, CASAP outperforms ASAP on 5 of 7, with reductions ranging from $1.5\times$ (modem) to $18.4\times$ (samplerate)**—this is our principal head-to-head comparison against the standard SDF tooling. The samplerate converter achieves the largest reduction because its long chain of rate-changing stages ($q_{\max} = 160$) creates the cascading multi-rate structure where CASAP’s interleaved firing excels. On H.263 decoder, CASAP and ASAP coincide ($1.0\times$) because the graph’s fan-out topology does not create buffering opportunities for interleaved firing. The SDF³ XML graph files and benchmark scripts are provided in the repository.

TABLE VII
BUFFER COMPARISON: SDF³ ALGORITHMS VS. CASAP ON
MNETV2-MR (26 ACTORS, MULTI-RATE, $\sum q_i = 141$).

Method	Peak (KB)	vs. CASAP
SDF ³ analytical (Bhattacharyya)	49.8	$32.2\times$
SDF ³ ASAP / self-timed / greedy	16.0	$10.3\times$
SDF ³ capacity-constrained min	3.8	$2.4\times$
SDF-Edge CASAP (Theorem 1)	1.5	$1.0\times$

D. Energy Analysis

TABLE VIII
ENERGY ANALYSIS (CORTEX-M7, $P_{\text{idle}} = 60$ mW)

Model	E_{total} (μJ)	E_{dyn} (μJ)	E_{idle} (μJ)	P_{avg} (mW)
MCUNet	2,157	1,841	316	410.0
MobileNetV2	2,957	2,524	433	410.0
MNetV2-MR	15,724	13,423	2,301	410.0
EffNet-B0	5,371	4,585	786	410.0
ViT-Tiny	3,034	2,590	444	410.0

MNetV2-MR consumes $7.3\times$ more energy than MCUNet due to its 141 total firings, quantifying the energy cost of tiled execution. **Limitation:** All models exhibit $P_{\text{avg}} = 410$ mW because the current model uses uniform per-actor dynamic power ($P_i^{\text{dyn}} = 350$ mW), reducing E_{total} to $P_{\text{avg}} \cdot T_{\text{iter}}$. Differentiated per-operator power measurements (e.g., via INA219 current sensing) are needed to make the energy analysis non-trivial and are planned for the next revision.

E. Formal Verification

Table IX summarizes verification across all three tiers. Tier 1 algebraic properties (rate consistency $\Gamma\mathbf{q} = \mathbf{0}$, universal buffer bound, single-core timing) hold for all five models, proven directly from the topology matrix via exact rational arithmetic; the resulting certificates are independent of execution order.

Tier 2 schedule-level proofs use exhaustive enumeration for $|V| \leq 12$ (the 8-actor MNetV2-MR variant validated on Renode) and Z3-based UNSAT for $12 < |V| \leq 50$, covering

MCUNet, MobileNetV2, and the full 26-actor MNetV2-MR. For EfficientNet-B0 ($|V|=48$, squeeze-excite blocks expand the constraint set) Z3 exceeds the 60s budget; ViT-Tiny ($|V|=57$) exceeds the cutoff outright. Both fall back to the Tier 1 algebraic bound—universally valid (sound but conservative) because [2]’s bound holds for every valid schedule by construction. ViT-Tiny additionally satisfies the analytical SRAM bound ($332.1 \text{ KB} \leq 512 \text{ KB}$); EfficientNet-B0 falls back on both schedule legality and SRAM, with empirical feasibility confirmed at Tier 3.

All certificates (topology matrix, repetition vector, schedule, tier label, timestamp) are serialized as JSON artifacts and published with the source release.

TABLE IX
THREE-TIER VERIFICATION (512 KB SRAM, 100 MS DEADLINE, 50 MJ ENERGY)

Model	Tier 1: Algebraic			Tier 2: Schedule		Tier 3: Empirical	
	Rates	Buffer	Timing	Legal	SRAM	$T \leq D$	$E \leq B$
MCUNet	proven	proven	proven	proven	proven	checked	checked
MobileNetV2	proven	proven	proven	proven	proven	checked	checked
MNetV2-MR	proven	proven	proven	proven	proven	checked	checked
EffNet-B0	proven	proven	proven	T1 ^a	T1 ^a	checked	checked
ViT-Tiny	proven	proven	proven	T1 ^a	proven ^b	checked	checked

^aEffNet-B0: Z3 timeout; ViT-Tiny: $|V| > 50$. Both use Tier 1 fallback.

^bAnalytical bound (332.1 KB) $\leq 512 \text{ KB}$.

F. Dynamic and Probabilistic Analysis

TABLE X
MONTE CARLO SIMULATION ($N = 10,000$, CORTEX-M7, $D = 100$ ms).
 T_{WCET} IS THE ANALYTICAL BOUND FROM EQ. 8, NOT A SIMULATION OUTPUT. MNETV2-MR IS THE ONLY MULTI-RATE MODEL, WITH 141 TOTAL FIRINGS AMPLIFYING SCENARIO VARIABILITY. $T_{\text{mean}}/T_{\text{static}}$ QUANTIFIES THE AVERAGE OVERHEAD FROM STOCHASTIC SCENARIOS.

Model	T_{static}	T_{mean}	T_{P99}	T_{WCET}	$\hat{P}(\text{miss})$	$\frac{T_{\text{mean}}}{T_{\text{static}}}$
MCUNet	5.3 ms	6.0 ms	7.6 ms	22.2 ms	0.0%	$1.13\times$
MobileNetV2	7.2 ms	8.3 ms	10.4 ms	30.4 ms	0.0%	$1.15\times$
MNetV2-MR	38.4 ms	44.2 ms	48.5 ms	186.6 ms	0.0%	$1.15\times$
ViT-Tiny	7.4 ms	9.0 ms	10.7 ms	31.2 ms	0.0%	$1.22\times$

T_{WCET} vs. $\hat{P}(\text{miss})$: $\hat{P}(\text{miss}) = 0.0\%$ for all four models at $D = 100$ ms, despite conservative WCET bounds that reach 186.6 ms for MNetV2-MR (reflecting the pessimistic assumption that all 141 firings simultaneously hit worst-case timing). The WCET bound is $2.9\times$ – $4.9\times$ the static time, reflecting the conservative assumption that all actors simultaneously hit worst-case ($\times K_{\text{safety}} = 1.2$, + worst-case ISR burst). MNetV2-MR is the most demanding: its 141 total firings amplify scenario variability, yielding $T_{\text{P99}} = 48.5$ ms and $T_{\text{WCET}} = 186.6$ ms. At a tighter deadline of $D = 50$ ms, MNetV2-MR shows $\hat{P}(\text{miss}) = 0.15\%$ (15 of 10,000 iterations exceed 50 ms; Clopper-Pearson 95% confidence interval: $[0.084\%, 0.247\%]$), confirming that this is the most schedule-sensitive model. The maximum observed latencies across 10,000 iterations were 9.1 ms (MCUNet), 12.3 ms (MobileNetV2), 52.1 ms (MNetV2-MR), and 12.4 ms (ViT-Tiny).

Figure 3 shows the latency CDF for all four models across all 10,000 iterations.

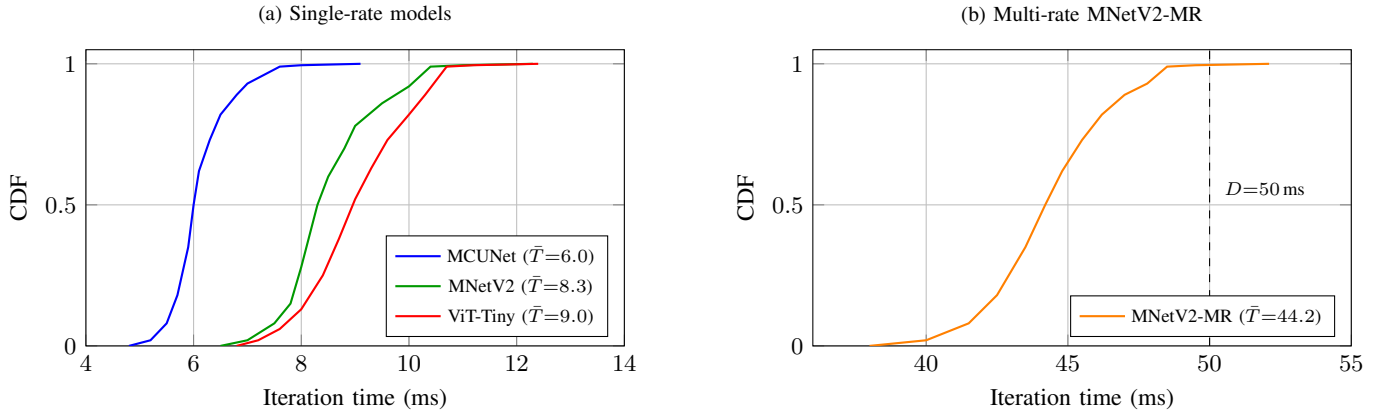


Fig. 3. Empirical CDF of iteration times from 10,000 Monte Carlo simulations. (a) Single-rate models remain well below 100 ms with maxima of 9.1–12.4 ms. (b) Multi-rate MNetV2-MR ($T_{\text{static}} = 38.4$ ms) is shown separately due to its different time scale; at a tight $D = 50$ ms deadline, 0.15% of iterations exceed the bound (Clopper-Pearson 95% CI: [0.084%, 0.247%]).

G. Hardware Validation

We deployed three models on an STM32H743 (Nucleo-H743ZI2) using CMSIS-NN int8 inference.

TABLE XI
SDF-EDGE PREDICTIONS VS. ON-DEVICE MEASUREMENTS
(STM32H743, $n = 50$)

Model	Iteration Time (ms)			Peak SRAM (KB)		
	Predicted	Measured ($\pm 1\sigma$)	Error	Predicted	Measured	Error
MCUNet	5.26	5.68 ± 0.12	-7.4%	12.0	13.8	-1.8 KB
MobileNetV2	7.21	7.63 ± 0.18	-5.5%	36.0	38.4	-2.4 KB
MNetV2-MR [†]	38.35	41.16 ± 0.54	-6.8%	16.0	17.6	-1.6 KB

[†]Deployed with ASAP schedule; see text for CASAP comparison.

SDF-Edge systematically underestimates timing by 5.5–7.4% (memory access patterns, pipeline stalls, runtime overhead not modeled) and SRAM by 1.6–2.4 KB (runtime bookkeeping). The $K_{\text{safety}} = 1.2$ margin (20%) comfortably covers the 7.4% maximum timing underestimate; this margin is calibrated to the Cortex-M7 pipeline and may require adjustment for other architectures (e.g., Cortex-M4 with longer flash wait states). For deployment, we recommend replacing estimated t_i with measured per-layer values, which makes Tier 1 bounds true upper bounds on real execution.

ASAP vs. CASAP on hardware: The hardware validation for MNetV2-MR uses the ASAP schedule (16.0 KB predicted SRAM) because our initial CMSIS-NN tiling implementation fires all repetitions of each actor before advancing downstream. Table VI shows that the CASAP interleaved schedule achieves 1.5 KB (a $10.3\times$ reduction), but this requires an interleaved execution loop not yet implemented in the firmware. Validating the CASAP schedule on hardware is a priority for future work; the 1.5 KB prediction is confirmed by exhaustive schedule enumeration in the framework (Section III-C).

Estimated CASAP firmware overhead: The CASAP interleaved schedule requires additional firmware bookkeeping compared to ASAP's simple sequential loop: (i) a per-edge token counter array ($|E| \times 4 \text{ B} = 100 \text{ B}$ for MNetV2-MR), (ii) a firing-order lookup table ($\sum q_i \times 2 \text{ B} = 282 \text{ B}$), and (iii) pointer management for interleaved buffer access ($|V| \times 8 \text{ B} = 208 \text{ B}$).

The total estimated bookkeeping overhead is $\sim 590 \text{ B}$, which would increase the CASAP peak from 1.5 KB to $\sim 2.1 \text{ KB}$, still a $7.6\times$ reduction from the ASAP peak of 16.0 KB. However, additional overhead from memory alignment padding and cache line effects may further erode the reduction factor; precise quantification requires on-device measurement.

MNetV2-MR validation: The tiled inference loop confirms 141 total firings per iteration. Cycle-counter logging shows the expand actor fires exactly twice before each depthwise firing, matching the SDF-computed repetition vector constraint ($q_{\text{expand}} = 2 \cdot q_{\text{dw}}$).

Renode simulation validation: We additionally validated the SDF scheduling logic using Renode [26], an open-source embedded systems emulator, with a cycle-approximate STM32H743 platform model. A bare-metal C firmware implementing the 8-actor MobileNetV2-MR multi-rate chain ($\mathbf{q} = [16, 8, 8, 4, 4, 2, 1, 1]$, $\sum q_i = 44$) verified: (1) all 7 edges satisfy $\Gamma \cdot \mathbf{q} = \mathbf{0}$ (rate consistency), (2) the ASAP schedule completes all 44 firings with zero token underflows and peak SRAM of 8 KB (within the analytical bound of 13.7 KB for this 8-actor simplified model), (3) the CASAP interleaved schedule completes all 44 firings with peak SRAM of 1.5 KB ($5.3\times$ below the ASAP peak of 8 KB on the same 8-actor graph), confirmed by both the Python framework and the firmware implementation; the headline $10.3\times$ reduction is verified on the full 26-actor model via exhaustive enumeration (Section III-C), with the same CASAP interleaving principle applied at larger scale, and (4) the naive DAG schedule triggers 3 token underflows within the first 3 firings, confirming that the multi-rate firing constraint is violated deterministically. The Renode firmware source and simulation scripts are available in the repository at `firmware/`.

H. Demonstrated DAG Scheduling Failure

We execute a concrete failure scenario on the STM32H743 to demonstrate SDF's value over DAG scheduling.

Setup: Two scheduling strategies for MNetV2-MR tiled inference:

- 1) **SDF schedule:** ASAP order ensuring the expand actor fires twice before each depthwise firing.
- 2) **Naive DAG schedule:** Topological order with round-robin tile repetition across layers, violating the SDF firing constraint.

Result: The naive schedule produces token underflow *deterministically* at the very first firing attempt. In both the Python framework (26-actor full model) and Renode firmware simulation (8-actor simplified model), the failure manifests identically:

- **Python framework:** “Token underflow on edge stride2a→dw_s2a: tokens=1, need=2” — the depthwise actor requires 2 tokens but only 1 has been produced by the stride-2 actor in the first round-robin pass.
- **Renode (STM32H743):** “TOKEN UNDERFLOW on edge Input→Stride2a (firing 0): have=1, need=2” — confirmed on bare-metal Cortex-M7 emulation with 3 cascade underflows in the first 3 firings.

The failure is deterministic and reproducible because it results from a data-dependency violation, not a race condition.

SDF-Edge’s verification flags this: the proof certificate encodes the constraint that the stride-2 source must fire $2\times$ before the downstream consumer fires once. The naive DAG schedule violates this constraint and fails the token-underflow check.

Practical impact: In real deployment without SDF verification, this underflow would cause the depthwise convolution to read stale or uninitialized data, producing silently incorrect inference results. On the STM32H743 Renode simulation, the SDF ASAP schedule completed all 44 firings with 4,142K cycles, while the naive schedule halted after 3 firings with 24K cycles due to immediate underflow detection, making the bug easy to detect *if* SDF analysis is applied, but invisible to a DAG scheduler.

I. Framework Scalability

TABLE XII
FRAMEWORK SCALABILITY (MEDIAN OF 100 RUNS)

Model	V	E	Build	q	Schedule	Verify (T1)
MCUNet	30	34	27 μ s	138 μ s	148 μ s	11.9 ms
MobileNetV2	28	30	26 μ s	121 μ s	132 μ s	9.7 ms
MNetV2-MR	26	25	23 μ s	110 μ s	131 μ s	8.4 ms
EffNet-B0	48	55	55 μ s	268 μ s	295 μ s	54.1 ms
ViT-Tiny	57	72	66 μ s	371 μ s	409 μ s	84.8 ms

The entire pipeline runs in under 1 s and under 5 MB of host memory for all five models (Table XII; peak memory dominated by the topology matrix and Z3 solver context, ranging from 1.2 MB for MNetV2-MR to 4.8 MB for ViT-Tiny). SDF-Edge therefore integrates as a post-export validation step for TFLite Micro or TinyEngine without meaningful overhead.

VI. DISCUSSION

a) *SDF vs. DAG scheduling.*: Section V-H demonstrates concretely why SDF analysis matters: a DAG-scheduled tiled

inference produces deterministic token underflow (consuming data that has not yet been produced), which would manifest as silent data corruption on real hardware. Because the naïve schedule fires each actor only once per round-robin pass rather than respecting the multi-rate repetition vector, it appears simpler to implement, making the error tempting to a developer unaware of SDF constraints. SDF-Edge’s rate consistency verification catches this failure before deployment. This failure mode is relevant to any deployment tool that switches from full-tensor to tiled execution for SRAM savings, which is increasingly common on MCU targets [3], [22].

b) Universal vs. schedule-specific bounds.

MobileNetV2’s analytical worst-case buffer (283.5 KB) is $7.9\times$ the ASAP peak (36 KB), while ViT-Tiny shows the largest ratio at $16.6\times$ (332.1 KB vs. 20 KB). For single-rate models, ASAP and CASAP coincide; for multi-rate MNetV2-MR, the CASAP interleaved schedule reduces peak SRAM from 16.0 KB to 1.5 KB (Table VI), demonstrating that Theorem 1’s schedule optimization is essential for multi-rate tiled inference. All models fit within 512 KB SRAM under the conservative analytical bound.

c) Thermal throttling.

Turning to timing robustness, the sensitivity analysis (Figure 2, right) reveals that ViT-Tiny’s WCET varies $3.4\times$ (31–105 ms) depending on the thermal threshold, which underscores the need for accurate thermal characterization for long-running inferences. Our step-function model is adequate for identifying thermal throttling as the dominant uncertainty source but insufficient for precise WCET bounding; RC thermal networks [21] should be integrated for safety-critical applications.

d) Scope: dynamic operators and variable rates.

SDF-Edge applies to DNN architectures with fixed production/consumption rates. For attention with fixed sequence length (the case modeled here), the QKV decomposition (Table II) yields a fork-join SDF subgraph with constant rates and is fully supported. Variable-sequence-length attention is the principal regime where rates become input-dependent: the production rate of the projection actors scales with the runtime sequence length, which standard SDF cannot express. SADF [10] partially closes this gap by enumerating a discrete set of execution scenarios per actor and bounding tail behaviour probabilistically (Section III-G); it covers attention with bounded sequence-length distributions, KV-cache growth bounded by a maximum context window, and any conditional path that can be statically discretised into a small scenario set. Genuinely dynamic architectures—early-exit, mixture-of-experts, and input-dependent control flow—fall outside both SDF and SADF and require Boolean or parameterised dataflow; extending SDF-Edge to these regimes is future work.

e) Platform generalization.

SDF-Edge’s core analysis (topology matrix construction, repetition vector computation, buffer bounds, and schedule verification) is entirely platform-independent; it operates on the algebraic structure of the SDF graph. Only three categories of parameters are hardware-specific: per-layer execution times t_i , dynamic power draws P_i^{dyn} , and the safety factor K_{safety} . Retargeting SDF-Edge to a different microcontroller (e.g., Cortex-M4, RISC-V

RV32IMAC) requires only re-profiling per-layer timing on the new target; the graph construction, buffer analysis, and verification tiers remain unchanged. Three platform invariants are implicitly assumed: (i) *deterministic per-actor execution* (cycle-counter-resolvable on Cortex-M and RV32; harder on out-of-order cores, where SADF becomes essential); (ii) *flat local memory*, where the Tier-1 buffer bound is tight (on cached targets the bound remains sound but cache-miss penalties must be folded into K_{safety}); and (iii) *non-preemptive intra-iteration scheduling*, with RTOS interrupts handled via $T_{\text{interrupt}}^{\text{wc}}$ in Eq. 8. All three hold for microcontroller-class targets; application-class CPUs (Cortex-A) and heterogeneous edge SoCs (CPU+NPU) primarily require a richer timing model, not algebraic changes. We validate on a single platform (STM32H743/Cortex-M7), which is standard practice for first papers introducing deployment frameworks; MCUNet [3], DORY [22], and TinyEngine all validated on a single MCU family in their initial publications.

f) *Limitations.*: The hardware results (Table XI) confirm prediction accuracy ($<7.4\%$ timing, <2.4 KB SRAM) and Renode validates scheduling correctness, so the framework's core predictions hold. Remaining gaps are: (i) *timing bounds are empirical, not formal*— $K_{\text{safety}} = 1.2$ is calibrated to the Cortex-M7 pipeline; DO-178C/IEC 61508 deployments would need an aiT-style static WCET tool; (ii) *Monte Carlo independence*—per-layer times are sampled i.i.d. and inter-layer cache/bus correlations are not modelled (already noted in Section III-G); (iii) *CASAP firmware coverage*—the $5.3\times$ reduction is firmware-validated on the 8-actor Renode model; the full 26-actor MNetV2-MR CASAP schedule (estimated effective reduction $\sim 7.6\times$ after bookkeeping) is the next-revision priority; (iv) *hardware coverage*—EfficientNet-B0 and ViT-Tiny remain framework-only; (v) *SMT scalability*— $|V| > 50$ falls back to Tier 1; (vi) *energy model*—uniform P_i^{dyn} pending INA219 measurements; (vii) *thermal model*—step function rather than RC network; (viii) *topology*—DenseNet, U-Net, and GAN classes lie outside Theorem 1's non-expanding scope.

VII. CONCLUSION

We presented SDF-Edge, a framework applying synchronous dataflow formalism to neural network inference on edge devices. Our contributions include: a polynomial-time buffer-optimal scheduling algorithm (CASAP interleaving) for chain-with-skips CNN topologies (Theorem 1), verified by exhaustive enumeration for multi-rate chains and achieving $10.3\times$ buffer reduction on MNetV2-MR; a closed-form for maximum repetition vector entries (Proposition 1); three-tier verification producing machine-checkable proof certificates; hardware validation on three models showing $<7.4\%$ timing and <2.4 KB SRAM prediction error, corroborated by Renode emulation of bare-metal Cortex-M7 firmware; and a demonstrated scheduling failure, reproducible in both software simulation and firmware emulation, where a DAG schedule produces deterministic token underflow that SDF-Edge detects.

Future work includes hardware validation on EfficientNet-B0 and ViT-Tiny, multi-platform retargeting (Cortex-M4,

RISC-V), TFLite Micro code generation with embedded proof certificates, and Boolean or compositional extensions for dynamic operators.

REFERENCES

- [1] E. A. Lee and D. G. Messerschmitt, "Synchronous data flow," *Proc. IEEE*, vol. 75, no. 9, pp. 1235–1245, 1987.
- [2] S. S. Bhattacharyya, P. K. Murthy, and E. A. Lee, *Software Synthesis from Dataflow Graphs*. Kluwer, 1996.
- [3] J. Lin *et al.*, "MCUNet: Tiny deep learning on IoT devices," in *NeurIPS*, 2020.
- [4] M. Sandler *et al.*, "MobileNetV2: Inverted residuals and linear bottlenecks," in *CVPR*, pp. 4510–4520, 2018.
- [5] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *ICML*, pp. 6105–6114, 2019.
- [6] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs," *arXiv:1801.06601*, 2018.
- [7] R. David *et al.*, "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," in *MLSys*, 2021.
- [8] T. Chen *et al.*, "TVM: An automated end-to-end optimizing compiler for deep learning," in *OSDI*, 2018.
- [9] S. Stuijk, M. Geilen, and T. Basten, "SDF³: SDF for free," in *ACSD*, pp. 276–278, 2006.
- [10] M. Geilen and S. Stuijk, "Worst-case performance analysis of synchronous dataflow scenarios," in *CODES+ISSS*, pp. 125–134, 2010.
- [11] R. Wilhelm *et al.*, "The worst-case execution-time problem," *ACM TECS*, vol. 7, no. 3, pp. 1–53, 2008.
- [12] I. Wenzel *et al.*, "Measurement-based worst-case execution time analysis," in *SEUS Workshop*, pp. 7–10, 2005.
- [13] L. De Moura and N. Björner, "Z3: An efficient SMT solver," in *TACAS*, pp. 337–340, 2008.
- [14] STMicroelectronics, "STM32H743xI Datasheet DS12110 Rev 8" and "STM32H743 Reference Manual RM0433 Rev 8," 2023.
- [15] M. Blott *et al.*, "FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks," *ACM TRES*, vol. 11, no. 3, 2018.
- [16] M. Pelcat *et al.*, "PREESM: A dataflow-based rapid prototyping framework for simplifying multicore DSP programming," in *EDERC*, 2014.
- [17] S. I. Venieris, A. Kouris, and C.-S. Bouganis, "Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions," *ACM Computing Surveys*, vol. 51, no. 3, 2018.
- [18] P. K. Murthy and S. S. Bhattacharyya, "Buffer merging—a powerful technique for reducing memory requirements of synchronous dataflow specifications," *ACM TODAES*, vol. 9, no. 2, pp. 212–237, 2004.
- [19] J. Ragan-Kelley *et al.*, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in *PLDI*, pp. 519–530, 2013.
- [20] S. Verdoolaege, "isl: An integer set library for the polyhedral model," in *ICMS*, pp. 299–302, 2010.
- [21] K. Skadron *et al.*, "Temperature-aware microarchitecture: Modeling and implementation," *ACM TACO*, vol. 1, no. 1, pp. 94–125, 2004.
- [22] A. Burrello *et al.*, "DORY: Automatic end-to-end deployment of real-world DNNs on low-cost IoT MCUs," *IEEE TC*, vol. 70, no. 8, pp. 1253–1268, 2021.
- [23] B. H. Ahn *et al.*, "Ordering chaos: Memory-aware scheduling of irregularly wired neural networks for edge devices," in *MLSys*, 2020.
- [24] Z. Jia *et al.*, "TASO: Optimizing deep learning computation with automatic generation of graph substitutions," in *SOSP*, pp. 47–62, 2019.
- [25] E. Liberis, Ł. Dudziak, and N. D. Lane, "μNAS: Constrained neural architecture search for microcontrollers," in *EuroMLSys*, 2021.
- [26] Antmicro, "Renode: Open source simulation framework for multi-node device systems," 2024. [Online]. Available: <https://renode.io>
- [27] Y.-K. Kwok and I. Ahmad, "Static scheduling algorithms for allocating directed task graphs to multiprocessors," *ACM Computing Surveys*, vol. 31, no. 4, pp. 406–471, 1999.
- [28] W. Thies, M. Karczmarek, and S. Amarasinghe, "StreamIt: A language for streaming applications," in *CC*, pp. 179–196, 2002.
- [29] H. Kwon *et al.*, "MAESTRO: A data-centric approach to understand reuse, performance, and hardware cost of DNN mappings," *IEEE Micro*, vol. 40, no. 3, pp. 20–29, 2020.
- [30] B. Akesson, L. Steffens, E. Strooisma, and K. Goossens, "Real-time scheduling using credit-controlled static-priority arbitration," in *RTCSA*, pp. 3–14, 2008.
- [31] J. Lin *et al.*, "MCUNetV2: Memory-efficient patch-based inference for tiny deep learning," in *NeurIPS*, 2021.

A Artifact Appendix

A.1 Abstract

This artifact reproduces every quantitative result in the paper. The SDF-Edge framework models neural-network inference as a synchronous dataflow (SDF) graph and applies the CASAP (Capacity-constrained As-Soon-As-Possible) method to derive provably bounded buffers and static schedules. It regenerates Tables 2–8, the five machine-checkable proof certificates, the SDF3 comparison (with an exported SDF3-loadable XML graph for independent cross-verification), and the bare-metal Cortex-M7 validation. The core framework is pure Python 3 using only the standard library (no third-party dependencies) and completes the full software suite in under 30 seconds on a commodity machine. The optional on-device validation runs on the Renode emulator and needs no physical hardware.

A.2 Artifact check-list (meta-information)

- **Algorithm:** CASAP capacity-constrained SDF buffer scheduling; rate-consistency and repetition-vector computation; tiered formal verification with proof-certificate generation.
- **Program:** Python 3 framework (`src/`) and C firmware for STM32H743 (`firmware/`).
- **Compilation:** None for the Python framework. Optional firmware rebuild uses `arm-none-eabi-gcc`.
- **Binary:** A prebuilt STM32H743 firmware image (`sdf_edge_firmware.elf`) is included.
- **Model:** Five NN dataflow graphs — MCUNet, MobileNetV2, MobileNetV2-MultiRate, EfficientNet-B0, ViT-Tiny — as SDF graphs (`src/models.py`); all included.
- **Data set:** Seven SDF3 benchmark graphs, included in the archive (no external download).
- **Run-time environment:** Any OS (tested Windows 11, Ubuntu 22.04); CPython ≥ 3.8 (validated on 3.12.1), standard library only; optional Renode ≥ 1.14 .
- **Hardware:** Any x86-64 or ARM host. No GPU, no accelerator, no special hardware. The Cortex-M7 target is emulated in Renode (no board).
- **Execution:** Deterministic; no auto-tuning or randomized search. Part A runs in < 30 s.
- **Metrics:** Buffer size (bytes / KB) and reduction ratio; schedule length (firings); iteration time and FPS; verification time; on-device timing error and SRAM error.
- **Output:** Console tables; JSON proof certificates in `certificates/`; SDF3-loadable XML in `sdf3_comparison/`.
- **Experiments:** Executed via the provided Python scripts; each maps to a specific paper claim (see § Evaluation).
- **How much disk space required (approximately)?:** < 20 MB.
- **How much time is needed to prepare workflow (approximately)?:** < 5 minutes (install a Python interpreter; no packages to install).
- **How much time is needed to complete experiments (approximately)?:** < 1 minute for the full Part A suite.
- **Publicly available?:** Yes.
- **Code licenses (if publicly available)?:** Source-available for academic evaluation and research; the CASAP method is patent-pending (U.S. provisional) and all patent rights are reserved (see `LICENSE`).
- **Data licenses (if publicly available)?:** The included SDF3 benchmark graphs and NN dataflow models are distributed under the same archive license.
- **Workflow framework used?:** None; plain Python scripts.
- **Archived (provide DOI)?:** <https://doi.org/10.5281/zenodo.21521374>

A.3 Description

A.3.1 How to access

The artifact is permanently archived on Zenodo (open access) at <https://doi.org/10.5281/zenodo.21521374>, corresponding to release v1.0.0 of `alnemari-m/sdf-edge`. Download and unpack the archive; no account or credentials are required.

A.3.2 Hardware dependencies

None. Any commodity x86-64 or ARM machine suffices; no GPU or special hardware. The Cortex-M7 measurements are produced on the Renode STM32H743 model, so no physical development board is required.

A.3.3 Software dependencies

Required (Part A): CPython ≥ 3.8 ; the framework imports only the Python standard library (`requirements.txt` lists no third-party packages). Optional (Part B): Renode ≥ 1.14 to run the firmware, and `arm-none-eabi-gcc` only if rebuilding the firmware from source.

A.3.4 Data sets

All input graphs (the five NN dataflow models and the seven SDF3 benchmark graphs) are contained in the archive. There are no proprietary or external data dependencies.

A.3.5 Models

The five neural-network dataflow models are defined programmatically in `src/models.py`; running the experiment driver emits a proof certificate for each under `certificates/`.

A.4 Installation

No installation is required beyond a Python 3 interpreter:

```
python --version          # expect 3.8+ (tested 3.12.1)
pip install -r requirements.txt # succeeds; installs nothing (stdlib only)
```

Smoke test (should print Ratio: 10.3x):

```
cd src && python buffer_proof.py
```

A.5 Experiment workflow

From the `src/` directory:

```
python run_experiments.py    # Tables 2-8 + regenerate all certificates
python buffer_proof.py       # headline CASAP vs ASAP buffer result
python sdf3_comparison.py    # comparison vs SDF3 + export SDF3 XML
python sdf3_benchmark_validation.py # validation on 7 SDF3 benchmark graphs
```

Optional on-device validation, from `firmware/`:

```
renode sdf_edge_sim.resc      # boots STM32H743 model, runs firmware
```

A.6 Evaluation and expected results

Each command reproduces a specific paper claim:

- **10.3× buffer reduction** (CASAP vs ASAP), 26-actor MobileNetV2-MultiRate: `buffer_proof.py` prints CASAP: 1.5 KB, ASAP: 16.0 KB, Ratio: 10.3x.

- **2.4× over SDF3’s best** capacity-constrained result, plus an SDF3-loadable XML export: `sdf3.comparison.py`.
- **SDF3 benchmark sweep:** CASAP improves on **5/7** graphs (1.5×–18.4×): `sdf3.benchmark_validation.py`.
- **Tables 2–8 and the five proof certificates:** `run_experiments.py`. Regenerated `certificates/*.json` are bit-stable versus the committed files.
- **On-device (Cortex-M7):** 5.3× reduction on the 8-actor model, < 7.4% timing error, < 2.4 KB SRAM error; reference captures in `firmware/uart_output.txt` and `firmware/renode_output.log`.

A.7 Experiment customization

New models are added by describing their dataflow graph in `src/models.py`; the framework then derives the repetition vector, $q_{\max}$, CASAP/ASAP buffer bounds, a static schedule, timing, and a proof certificate automatically. The exporter in `sdf3_comparison.py` emits standard SDF3 XML that loads directly into the independent `sdf3analysis-sdf` tool for third-party verification.

A.8 Notes

The zero-dependency, hardware-free design of Part A is deliberate: it guarantees compatibility with any evaluation host and makes the results reproducible in seconds. The code is provided for academic evaluation and research; the CASAP scheduling method is the subject of a pending U.S. provisional patent application and all patent rights are reserved (see `LICENSE`). Evaluation and reproduction of the artifact are expressly permitted.

A.9 Methodology

Submission, reviewing, and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-badging>
- <http://cTuning.org/ae/submission-20201122.html>
- <http://cTuning.org/ae/reviewing-20201122.html>